

PL/SQL : TD Initiation au langage PL/pgSQL

Délio Duchesne-Brajon

SLAM

Exercice 1 :

```
SQL Shell (psql)
hhbdirect=# CREATE OR REPLACE FUNCTION calculer_longueur_max(chaine1 varchar, chaine2 varchar) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     long1 integer;
hhbdirect$#     long2 integer;
hhbdirect$# BEGIN
hhbdirect$#     long1 := length(chaine1);
hhbdirect$#     long2 := length(chaine2);
hhbdirect$#     IF long1 > long2 THEN
hhbdirect$#         RETURN long1;
hhbdirect$#     ELSE
hhbdirect$#         RETURN long2;
hhbdirect$#     END IF;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT calculer_longueur_max('bonjour', 'salut');
calculer_longueur_max
-----
7
(1 ligne)

hhbdirect=#
```

Exercice 2 :

1/ Version avec la boucle FOR

```
SQL Shell (psql)
hhbdirect=# CREATE OR REPLACE FUNCTION nb_occurrences_for(car char(1), chaine varchar, debut integer, fin integer) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     nb integer := 0;
hhbdirect$#     i integer;
hhbdirect$#     sous_chaine varchar;
hhbdirect$# BEGIN
hhbdirect$#     IF debut < 1 OR fin > length(chaine) OR debut > fin THEN
hhbdirect$#         RAISE EXCEPTION 'Intervalle invalide';
hhbdirect$#     END IF;
hhbdirect$#     sous_chaine := substr(chaine, debut, fin - debut + 1);
hhbdirect$#     FOR i IN 1..length(sous_chaine) LOOP
hhbdirect$#         IF substr(sous_chaine, i, 1) = car THEN
hhbdirect$#             nb := nb + 1;
hhbdirect$#         END IF;
hhbdirect$#     END LOOP;
hhbdirect$#     RETURN nb;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT nb_occurrences_for('e', 'superieur', 2, 6);
nb_occurrences_for
-----
1
(1 ligne)

hhbdirect=#
```

2/ Version avec la boucle LOOP / EXIT

```
SQL Shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION nb_occurrences_loop(caractere varchar, chaine varchar, debut integer, fin integer) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     nb integer := 0;
hhbdirect$#     i integer := 1;
hhbdirect$#     sous_chaine varchar;
hhbdirect$#     longueur integer;
hhbdirect$# BEGIN
hhbdirect$#     IF debut < 1 OR fin > length(chaine) OR debut > fin THEN
hhbdirect$#         RAISE EXCEPTION 'Intervalle invalide';
hhbdirect$#     END IF;
hhbdirect$#     sous_chaine := substr(chaine, debut, fin - debut + 1);
hhbdirect$#     longueur := length(sous_chaine);
hhbdirect$#     LOOP
hhbdirect$#         EXIT WHEN i > longueur;
hhbdirect$#         IF substr(sous_chaine, i, 1) = caractere THEN
hhbdirect$#             nb := nb + 1;
hhbdirect$#         END IF;
hhbdirect$#         i := i + 1;
hhbdirect$#     END LOOP;
hhbdirect$#     RETURN nb;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT nb_occurrences_loop('e', 'superieur', 2, 6);
  nb_occurrences_loop
-----
                1
(1 ligne)

hhbdirect=#
```

3/ Version avec la boucle WHILE

```
SQL Shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION nb_occurrences_while(caractere varchar, chaine varchar, debut integer, fin integer) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     nb integer := 0;
hhbdirect$#     i integer := 1;
hhbdirect$#     sous_chaine varchar;
hhbdirect$#     longueur integer;
hhbdirect$# BEGIN
hhbdirect$#     IF debut < 1 OR fin > length(chaine) OR debut > fin THEN
hhbdirect$#         RAISE EXCEPTION 'Intervalle invalide';
hhbdirect$#     END IF;
hhbdirect$#     sous_chaine := substr(chaine, debut, fin - debut + 1);
hhbdirect$#     longueur := length(sous_chaine);
hhbdirect$#     WHILE i <= longueur LOOP
hhbdirect$#         IF substr(sous_chaine, i, 1) = caractere THEN
hhbdirect$#             nb := nb + 1;
hhbdirect$#         END IF;
hhbdirect$#         i := i + 1;
hhbdirect$#     END LOOP;
hhbdirect$#     RETURN nb;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT nb_occurrences_while('e', 'superieur', 2, 6);
  nb_occurrences_while
-----
                1
(1 ligne)

hhbdirect=#
```

Exercice 3 :

```
SQL Shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION getNbJoursParMois(d date) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     dernier_jour date;
hhbdirect$# BEGIN
hhbdirect$#     dernier_jour := date_trunc('month', d) + interval '1 month' - interval '1 day';
hhbdirect$#     RETURN extract(day from dernier_jour);
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT getNbJoursParMois('2012-02-15');
getnbjoursparmois
-----
                29
(1 ligne)

hhbdirect=#
```

Exercice 4 :

```
SQL Shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION dateSqlToDatefr(d date) RETURNS varchar AS $$
hhbdirect$# BEGIN
hhbdirect$#     RETURN to_char(d, 'DD/MM/YY');
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT dateSqlToDatefr('2012-10-27');
datesqltodatefr
-----
27/10/12
(1 ligne)

hhbdirect=#
```

Exercice 5 :

```
SQL shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION getNomJour(d date) RETURNS varchar AS $$
hhbdirect$# DECLARE
hhbdirect$#     num_jour integer;
hhbdirect$# BEGIN
hhbdirect$#     num_jour := extract(dow from d);
hhbdirect$#     CASE num_jour
hhbdirect$#         WHEN 0 THEN RETURN 'Dimanche';
hhbdirect$#         WHEN 1 THEN RETURN 'Lundi';
hhbdirect$#         WHEN 2 THEN RETURN 'Mardi';
hhbdirect$#         WHEN 3 THEN RETURN 'Mercredi';
hhbdirect$#         WHEN 4 THEN RETURN 'Jeudi';
hhbdirect$#         WHEN 5 THEN RETURN 'Vendredi';
hhbdirect$#         WHEN 6 THEN RETURN 'Samedi';
hhbdirect$#     END CASE;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT getNomJour('2012-10-27');
   getnomjour
-----
   Samedi
(1 ligne)

hhbdirect=#
```

Exercice 6 :

1/ Nombre de clients débiteurs (clients possédant un compte au solde négatif)

```
SQL shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION nb_clients_debiteurs() RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     nb integer;
hhbdirect$# BEGIN
hhbdirect$#     SELECT count(DISTINCT p.num_client) INTO nb
hhbdirect$#     FROM posseder p
hhbdirect$#     JOIN compte c ON p.id_type = c.id_type AND p.num_compte = c.num_compte
hhbdirect$#     WHERE c.solde < 0;
hhbdirect$#     RETURN nb;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT nb_clients_debiteurs();
   nb_clients_debiteurs
-----
                        1
(1 ligne)

hhbdirect=#
```

2/ Nombre de clients habitant dans une ville (la ville est passée en paramètre)

```
SQL Shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION nb_clients_par_ville(ville varchar) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     nb integer;
hhbdirect$# BEGIN
hhbdirect$#     SELECT count(*) INTO nb FROM client WHERE adresse_client ILIKE '%' || ville || '%';
hhbdirect$#     RETURN nb;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT nb_clients_par_ville('LANNION');
  nb_clients_par_ville
-----
2
(1 ligne)

hhbdirect=#
```

Exercice 7 :

Insertion d'un client, le numéro est calculé automatiquement (dernier numéro + 1) et un message indique si l'enregistrement a réussi.

```
SQL Shell (psql)

hhbdirect=# CREATE OR REPLACE FUNCTION inserer_client(p_nom varchar, p_prenom varchar, p_adresse varchar, p_identifiant varchar, p_mdp varchar) RETURNS varchar AS $$
hhbdirect$# DECLARE
hhbdirect$#     nouveau_num integer;
hhbdirect$# BEGIN
hhbdirect$#     SELECT COALESCE(max(num_client), 0) + 1 INTO nouveau_num FROM client;
hhbdirect$#     INSERT INTO client (num_client, nom_client, prenom_client, adresse_client, identifiant_internet, mdp_internet)
hhbdirect$#     VALUES (nouveau_num, p_nom, p_prenom, p_adresse, p_identifiant, p_mdp);
hhbdirect$#     RETURN 'Client bien enregistré avec le numero ' || nouveau_num;
hhbdirect$# EXCEPTION
hhbdirect$#     WHEN OTHERS THEN
hhbdirect$#     RETURN 'Erreur : le client n''a pas été enregistré';
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT inserer_client('MARTIN', 'Luc', '3 Rue Verte, 35000 RENNES', 'Lmartin', 'Lmartin');
  inserer_client
-----
Client bien enregistré avec le numero 6
(1 ligne)

hhbdirect=#
```